Lab 7 – Fuzz Testing Report

Student Grades Application | Hypothesis Property-Based Testing

Name: Rohan Komperla

SRN: PES1UG24CS380

Section: G

Objective

This report documents fuzz testing of the student-grades application using Hypothesis and pytest. The functions tested are **average_marks()**, **parse_srn()** and **short_name()**.

1. Application execution

The application accepts comma-separated marks. The sample input **78,85,92** produces an average of **85.0**.

```
Refer to the lab handout for instructions.
PS C:\Users\Rohan\L6_L7_16_student_grades\Lab 7> python student_grades.py
==== Running average_marks ====
Enter marks as CSV (e.g. 78,85,92): 78,85,92
Average Marks: 85.0

==== Running parse_srn ====
Enter an SRN (e.g. PES1UG23CS123):
```

2. Failing fuzz tests

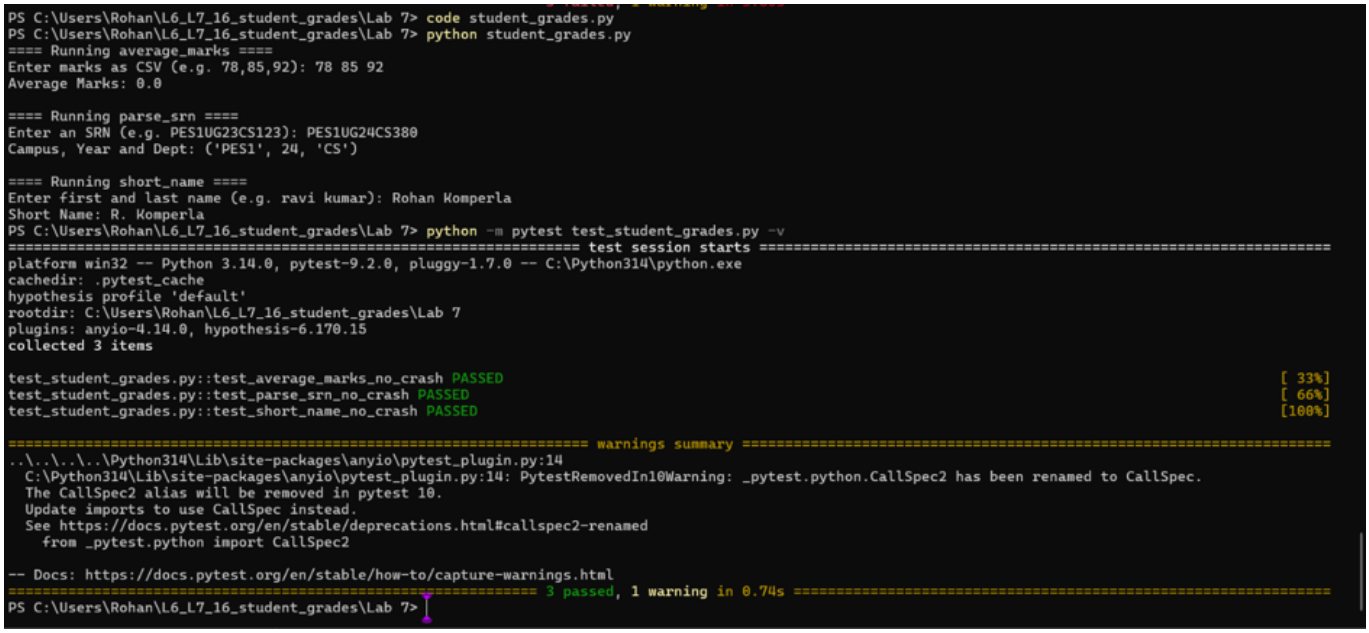
The initial Hypothesis run identified failures in all three functions. The displayed traceback includes an invalid **None** input to **short_name()**, which caused an AttributeError.

```
  | )
  | Explanation:
  |     These lines were always and only run by failing test cases:
  |         C:\Users\Rohan\L6_L7_16_student_grades\Lab 7\test_student_grades.py:26
  +---------------- 2 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\Rohan\L6_L7_16_student_grades\Lab 7\test_student_grades.py", line 25, in test_short_name_no_crash
  |     short_name(s)
  |     ^^^^^^^^^^^^^
  |   File "C:\Users\Rohan\L6_L7_16_student_grades\Lab 7\student_grades.py", line 9, in short_name
  |     first, last = full_name.split()
  |                   ^^^^^^^^^^^^^^^^
  | AttributeError: 'NoneType' object has no attribute 'split'
  |
  | During handling of the above exception, another exception occurred:
  |
  | Traceback (most recent call last):
  |   File "C:\Users\Rohan\L6_L7_16_student_grades\Lab 7\test_student_grades.py", line 27, in test_short_name_no_crash
  |     assert False, f"short_name crashed with: {e}"
  | AssertionError: short_name crashed with: 'NoneType' object has no attribute 'split'
  | assert False
  | Failing test case: test_short_name_no_crash(
  |     s=None,
  | )
  | Explanation:
  |     These lines were always and only run by failing test cases:
  |         C:\Users\Rohan\L6_L7_16_student_grades\Lab 7\test_student_grades.py:26
  +-----------------------------------
============================================================ warnings summary =============================================================
\..\..\..\Python314\Lib\site-packages\anyio\pytest_plugin.py:14
C:\Python314\Lib\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10Warning: _pytest.python.CallSpec2 has been renamed to CallSpec.
The CallSpec2 alias will be removed in pytest 10.
Update imports to use CallSpec instead.
See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
  from _pytest.python import CallSpec2

 Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================================== short test summary info ==========================================================
FAILED test_student_grades.py::test_average_marks_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_student_grades.py::test_parse_srn_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_student_grades.py::test_short_name_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
======================================================== 3 failed, 1 warning in 3.80s ========================================================
C:\Users\Rohan\L6_L7_16_student_grades\Lab 7>
```

3. Corrected application and final test run

After the input-handling fixes, the application completed using the sample SRN and name. The final pytest run reports all three fuzz tests as passed.



Final result: 3 passed, 1 warning.

The warning shown is a pytest/AnyIO deprecation notice and did not cause a test failure.

4. Corrected application source – student_grades.py

Complete corrected application source code:

```
def average_marks(marks_csv):
    if not isinstance(marks_csv, str) or not marks_csv.strip():
        return 0.0

    try:
        marks = [float(mark.strip()) for mark in marks_csv.split(",")]
        if not marks:
            return 0.0
        return round(sum(marks) / len(marks), 2)
    except (ValueError, TypeError):
        return 0.0


def parse_srn(srn):
    if not isinstance(srn, str) or len(srn) < 10:
        return None

    try:
        year = int(srn[6:8])
        if not srn[8:10].isalpha():
            return None
        return srn[:4], year, srn[8:10]
    except (ValueError, TypeError):
        return None


def short_name(full_name):
    if not isinstance(full_name, str):
        return ""

    parts = full_name.split()
    if len(parts) < 2:
        return ""

    first, last = parts[0], parts[-1]
    return first[0].upper() + ". " + last.capitalize()


def main():
    print("==== Running average_marks ====")
    user_input = input("Enter marks as CSV (e.g. 78,85,92): ")
    result = average_marks(user_input)
    print("Average Marks:", result)

    print("\n==== Running parse_srn ====")
    user_input = input("Enter an SRN (e.g. PES1UG23CS123): ")
    result = parse_srn(user_input)
    print("Campus, Year and Dept:", result)

    print("\n==== Running short_name ====")
    user_input = input("Enter first and last name (e.g. ravi kumar): ")
    result = short_name(user_input)
    print("Short Name:", result)


if __name__ == "__main__":
    main()
```

5. Hypothesis test source – test_student_grades.py

Completed Hypothesis tests covering all three application functions:

```
from hypothesis import given, strategies as st
from student_grades import average_marks, parse_srn, short_name


@given(st.text() | st.none())
def test_average_marks_no_crash(s):
    try:
        average_marks(s)
    except Exception as e:
        assert False, f"average_marks crashed with: {e}"


@given(st.text() | st.none())
def test_parse_srn_no_crash(s):
    try:
        parse_srn(s)
    except Exception as e:
        assert False, f"parse_srn crashed with: {e}"


@given(st.text() | st.none())
def test_short_name_no_crash(s):
    try:
        short_name(s)
    except Exception as e:
        assert False, f"short_name crashed with: {e}"
```